

[Definitions], page 3). The first word generally specifies a command to be executed, with the rest of the words being that command's arguments.

The return status (see Section 3.7.5 [Exit Status], page 48) of a simple command is its exit status as provided by the POSIX 1003.1 `waitpid` function, or `128+n` if the command was terminated by signal `n`.

3.2.3 Pipelines

A **pipeline** is a sequence of one or more commands separated by one of the control operators `|` or `&`.

The format for a pipeline is

```
[time [-p]] [!] command1 [ | or |& command2 ] ...
```

The output of each command in the pipeline is connected via a pipe to the input of the next command. That is, each command reads the previous command's output. This connection is performed before any redirections specified by *command1*.

If `|&` is the pipeline operator, *command1*'s standard error, in addition to its standard output, is connected to *command2*'s standard input through the pipe; it is shorthand for `2>&1 |`. This implicit redirection of the standard error to the standard output is performed after any redirections specified by *command1*, consistent with that shorthand.

If the reserved word `time` precedes the pipeline, Bash prints timing statistics for the pipeline once it finishes. The statistics currently consist of elapsed (wall-clock) time and user and system time consumed by the command's execution. The `-p` option changes the output format to that specified by POSIX. When the shell is in POSIX mode (see Section 6.11 [Bash POSIX Mode], page 116), it does not recognize `time` as a reserved word if the next token begins with a `-`. The value of the `TIMEFORMAT` variable is a format string that specifies how the timing information should be displayed. See Section 5.2 [Bash Variables], page 87, for a description of the available formats. Providing `time` as a reserved word permits the timing of shell builtins, shell functions, and pipelines. An external `time` command cannot time these easily.

When the shell is in POSIX mode (see Section 6.11 [Bash POSIX Mode], page 116), you can use `time` by itself as a simple command. In this case, the shell displays the total user and system time consumed by the shell and its children. The `TIMEFORMAT` variable specifies the format of the time information.

If a pipeline is not executed asynchronously (see Section 3.2.4 [Lists], page 11), the shell waits for all commands in the pipeline to complete.

Each command in a multi-command pipeline, where pipes are created, is executed in its own *subshell*, which is a separate process (see Section 3.7.3 [Command Execution Environment], page 46). If the `lastpipe` option is enabled using the `shopt` builtin (see Section 4.3.2 [The Shopt Builtin], page 78), and job control is not active, the last element of a pipeline may be run by the shell process.

The exit status of a pipeline is the exit status of the last command in the pipeline, unless the `pipefail` option is enabled (see Section 4.3.1 [The Set Builtin], page 74). If `pipefail` is enabled, the pipeline's return status is the value of the last (rightmost) command to exit with a non-zero status, or zero if all commands exit successfully. If the reserved word `!` precedes the pipeline, the exit status is the logical negation of the exit status as described

above. If a pipeline is not executed asynchronously (see Section 3.2.4 [Lists], page 11), the shell waits for all commands in the pipeline to terminate before returning a value. The return status of an asynchronous pipeline is 0.

3.2.4 Lists of Commands

A **list** is a sequence of one or more pipelines separated by one of the operators `;`, `&`, `&&`, or `||`, and optionally terminated by one of `;`, `&`, or a **newline**.

Of these list operators, `&&` and `||` have equal precedence, followed by `;` and `&`, which have equal precedence.

A sequence of one or more newlines may appear in a **list** to delimit commands, equivalent to a semicolon.

If a command is terminated by the control operator `&`, the shell executes the command asynchronously in a subshell. This is known as executing the command in the *background*, and these are referred to as *asynchronous* commands. The shell does not wait for the command to finish, and the return status is 0 (true). When job control is not active (see Chapter 7 [Job Control], page 125), the standard input for asynchronous commands, in the absence of any explicit redirections, is redirected from `/dev/null`.

Commands separated by a `;` are executed sequentially; the shell waits for each command to terminate in turn. The return status is the exit status of the last command executed.

AND and OR lists are sequences of one or more pipelines separated by the control operators `&&` and `||`, respectively. AND and OR lists are executed with left associativity.

An AND list has the form

```
command1 && command2
```

command2 is executed if, and only if, *command1* returns an exit status of zero (success).

An OR list has the form

```
command1 || command2
```

command2 is executed if, and only if, *command1* returns a non-zero exit status.

The return status of AND and OR lists is the exit status of the last command executed in the list.

3.2.5 Compound Commands

Compound commands are the shell programming language constructs. Each construct begins with a reserved word or control operator and is terminated by a corresponding reserved word or operator. Any redirections (see Section 3.6 [Redirections], page 41) associated with a compound command apply to all commands within that compound command unless explicitly overridden.

In most cases a list of commands in a compound command's description may be separated from the rest of the command by one or more newlines, and may be followed by a newline in place of a semicolon.

Bash provides looping constructs, conditional commands, and mechanisms to group commands and execute them as a unit.